

Supervised Learning - Regression

1	The Machine Learning Workflow	3
1.1	The steps	3
1.2	A worked example	3
2	Supervised vs. Unsupervised Learning	4
2.1	Supervised learning	4
2.2	Unsupervised learning	4
2.3	Side-by-side comparison	4
2.4	In Practice	4
3	Regression vs. Classification	5
3.1	Examples	5
3.2	In Practice	5
4	Linear Regression	5
4.1	Real-life example	6
4.2	In Practice	6
4.3	Reading the result	6
5	Ordinary Least Squares (OLS)	7
5.1	Why square the errors?	7
5.2	In Practice	7
6	Gradient Descent (Intuition)	8
6.1	The idea in four beats	8
6.2	In Practice	8
7	Multiple Linear Regression	9
7.1	Example	9
7.2	In Practice	9
7.3	Reading the coefficients	9

8 Polynomial Regression	10
8.1 Examples	10
8.2 In Practice	10
9 Overfitting and Underfitting	11
9.1 How to recognise them	11
9.2 Common causes	11
10 The Bias–Variance Tradeoff	12
10.1 Summary	12
11 Ridge Regression	13
11.1 In Practice	13
12 Lasso Regression	13
12.1 In Practice	14
13 Regression Evaluation Metrics	14
13.1 Mean Absolute Error (MAE)	15
13.2 Mean Squared Error (MSE)	15
13.3 Root Mean Squared Error (RMSE)	15
13.4 R^2 Score (Coefficient of Determination)	16
13.5 Which metric should I use?	17

1 The Machine Learning Workflow

Before writing a single line of model code, it helps to know the *whole journey* from raw data to a working predictor. We call this journey the **machine learning workflow**. Every project, big or small, follows roughly the same steps.

Key Idea

Machine learning turns **past data** into a **model** that can make good guesses about **new, unseen data**. The workflow is just the recipe we follow to build that model.

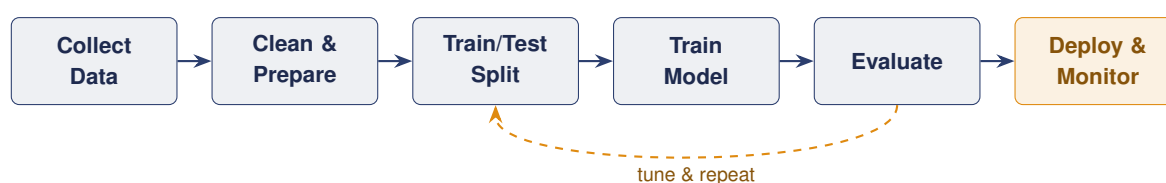
Real-Life Analogy

Building a model is like **cooking a new dish**. You decide what to cook (the goal), buy ingredients (collect data), wash and chop them (clean the data), follow the recipe (train the model), taste and adjust the salt (evaluate and tune), and finally serve it to guests (deploy). If guests complain later, you tweak the recipe (monitor and improve).

1.1 The steps

1. **Problem definition** — What are we predicting, and why?
2. **Data collection** — Gather the data we will learn from.
3. **Data cleaning** — Fix missing values, errors, and duplicates.
4. **Feature engineering** — Turn raw columns into useful inputs (encode, scale, combine).
5. **Train–test split** — Keep some data aside to test honestly.
6. **Model selection** — Pick a suitable algorithm.
7. **Model training** — Let the model learn the patterns.
8. **Model evaluation** — Measure how well it predicts.
9. **Hyperparameter tuning** — Adjust settings to improve results.
10. **Deployment** — Put the model to real use.
11. **Monitoring** — Keep checking it stays accurate over time.

The core of this loop can be pictured as a pipeline that we repeat until the results are good enough:



1.2 A worked example

Suppose a company wants to **predict house prices**. The workflow becomes: collect house records ➤ fix missing values ➤ convert text columns to numbers ➤ scale the numeric features ➤ train a regression model ➤ check the error ➤ deploy ➤ predict prices for new houses.

2 Supervised vs. Unsupervised Learning

Machine learning algorithms fall into two big families, and the difference is simply **whether the data comes with answers**.

Key Idea

Supervised learning learns from data that already has the correct answers (labels). **Unsupervised** learning explores data that has *no* answers and tries to find hidden structure on its own.

2.1 Supervised learning

The data includes both the *inputs* and the *correct output*. The model studies many examples of “input → output” and learns to reproduce that mapping.

Examples: *House features* → *house price*; *Email text* → *spam or not spam*.

2.2 Unsupervised learning

The data has *no* target column. There is no “right answer” to copy — the goal is to discover patterns or groups.

Examples: grouping customers by buying behaviour (segmentation); finding products that are similar to one another.

Real-Life Analogy

Supervised learning is like studying with **flash-cards that have the answer on the back**: you check yourself against the correct answer every time. Unsupervised learning is like being dropped in a **new city with no map** — nobody tells you the “right” neighbourhoods, so you group streets yourself based on what looks similar.

2.3 Side-by-side comparison

Supervised	Unsupervised
Data <i>has</i> labels (answers)	Data has <i>no</i> labels
Goal: predict an output	Goal: discover patterns
Tasks: Regression	Tasks: Clustering
Tasks: Classification	Tasks: Dimensionality reduction

2.4 In Practice

In supervised learning we separate the inputs X from the target y :

```

1         # Supervised: we have a target column called "Price"
2         X = df.drop("Price", axis=1)    # inputs (everything except Price)
3         y = df["Price"]                # target (the answer we want to
    predict)
```

```
4
5      # Unsupervised: there is no target, we use all the data as-is
6      X = df
7
```

3 Regression vs. Classification

Both are kinds of *supervised* learning (they both use labelled data). The difference is the **type of answer** we are predicting.

Key Idea

Regression predicts a *number* that can take any value (“how much?”). **Classification** predicts a *category* from a fixed set of options (“which one?”).

3.1 Examples

Regression (predict a number): salary, temperature, house price, tomorrow’s rainfall in mm.

Classification (predict a label): spam / not spam, diseased / healthy, pass / fail, fraud / genuine.

3.2 In Practice

The two families even share a similar code shape — you mostly swap the model:

```
1      # Regression -> predicts a continuous number
2      from sklearn.linear_model import LinearRegression
3
4      # Classification -> predicts a category
5      from sklearn.linear_model import LogisticRegression
6
```

4 Linear Regression

Linear regression is the simplest and most widely taught regression model. It assumes the relationship between input and output can be drawn as a **straight line**.

Key Idea

Linear regression finds the *straight line that best fits the data*, then uses that line to predict the output for any new input.

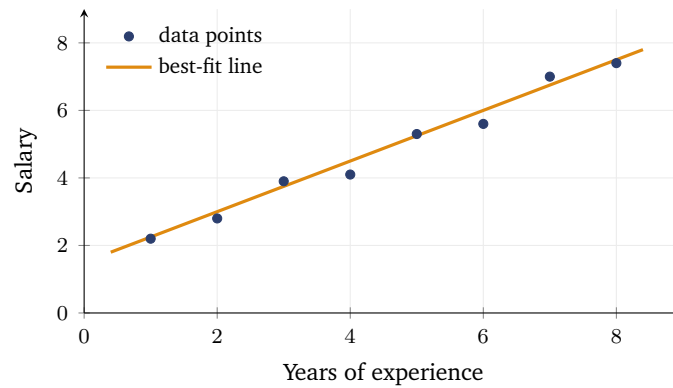
For one input feature, the line is described by:

$$y = \beta_0 + \beta_1 x$$

where β_0 is the **intercept** (where the line crosses the y -axis) and β_1 is the **slope** (how steeply y rises as x increases).

Real-Life Analogy

Imagine plotting people's *years of experience* against their *salary* as dots on paper. The dots roughly trend upward but are scattered. Linear regression lays a **ruler** across the cloud of dots in the position that sits as close to all of them as possible. To predict a new person's salary, you just read the height of the ruler at their experience.

**4.1 Real-life example**

Predict a person's salary from their years of experience: each extra year adds a roughly fixed amount to the predicted salary (that fixed amount is the slope β_1).

4.2 In Practice

```

1      from sklearn.linear_model import LinearRegression
2
3      model = LinearRegression()
4      model.fit(X_train, y_train)      # the model learns beta_0 and beta_1
5

```

4.3 Reading the result

After fitting, the learned numbers are stored on the model:

```

1      model.intercept_      # beta_0 -> the starting value when all inputs
    are 0
2      model.coef_          # beta_1 -> the slope(s): change in y per unit of
    x
3

```

When to Use It

The relationship looks roughly straight, you want a fast model, and you value being able to *explain* it (each coefficient has a clear meaning).

Try It Yourself

You fit a model and get `intercept_ = 25000` and `coef_ = [4000]`, with x in years. Write the line's equation in words, then predict the salary for someone with 6 years of experience.

5 Ordinary Least Squares (OLS)

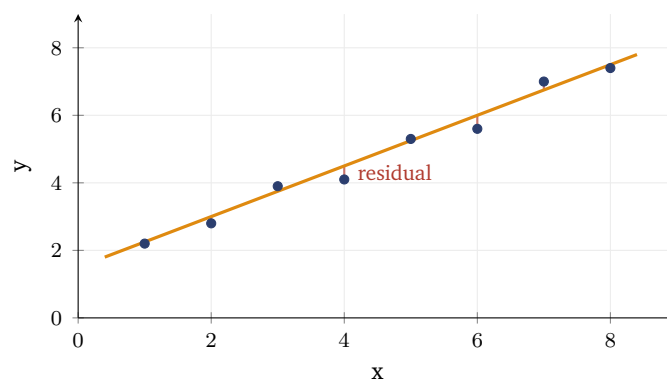
We said linear regression finds the “best” line — but what does *best* mean? The standard answer is **Ordinary Least Squares**.

Key Idea

OLS chooses the line that makes the **total squared error** as small as possible. The error for one point is its vertical distance from the line; OLS squares each of these and adds them up.

The vertical gap between an actual value y_i and the line's prediction $\hat{y}_i$ is the **residual**. OLS minimises the sum of squared residuals:

$$\text{minimise } \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$



5.1 Why square the errors?

Two good reasons: (1) squaring makes every error *positive*, so errors above and below the line do not cancel out; (2) it *penalises big misses far more* than small ones, pushing the line to avoid large mistakes.

5.2 In Practice

You do not have to do anything special — `scikit-learn`'s `LinearRegression` uses OLS by default:

```
1 from sklearn.linear_model import LinearRegression
2 model = LinearRegression() # solves the OLS problem internally
3
```

6 Gradient Descent (Intuition)

OLS has a neat formula that solves for the best line in one shot. But for very large datasets or more complex models, that direct formula becomes slow or impractical. **Gradient descent** is a general-purpose alternative that *creeps* toward the answer step by step.

Key Idea

Gradient descent starts with a random guess for the parameters and repeatedly nudges them in the direction that *reduces the error* the most, a little at a time, until the error stops shrinking.

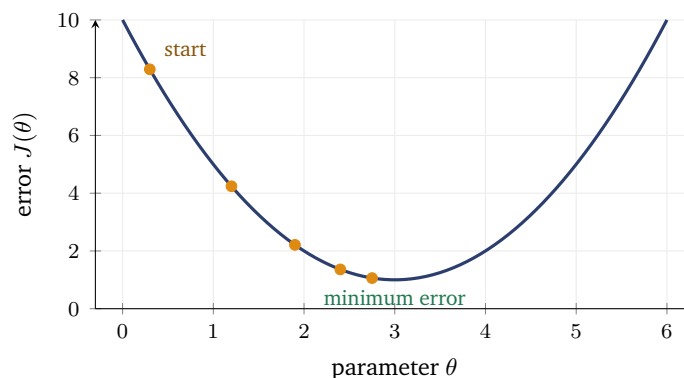
Real-Life Analogy

You are standing somewhere on a foggy hillside and want to reach the **bottom of the valley**. You cannot see far, so you feel the slope under your feet and take a step *downhill*. Repeat. Each step lowers you a bit more until the ground is flat — you have reached the lowest point. The size of each step is the **learning rate**: too big and you overshoot the valley; too small and it takes forever.

The update rule for a parameter θ is:

$$\theta = \theta - \alpha \frac{\partial J}{\partial \theta},$$

where J is the error (cost) and α is the learning rate (the step size).



6.1 The idea in four beats

Random weights ➤ measure the error ➤ step toward lower error ➤ repeat until the minimum.

6.2 In Practice

For plain linear regression you rarely call gradient descent yourself — it is the engine behind larger models such as neural networks. `scikit-learn` does expose a gradient-descent based regressor if you ever need it:

```
1 from sklearn.linear_model import SGDRegressor # uses gradient
   descent
2
3 model = SGDRegressor(learning_rate="optimal", max_iter=1000)
4 model.fit(X_train, y_train)
5
```


Watch Out

The **learning rate** is the make-or-break setting. Too large and the error bounces around and may never settle; too small and training crawls. When in doubt, start small and increase carefully.

7 Multiple Linear Regression

Real outcomes rarely depend on just one thing. **Multiple linear regression** simply lets the line use *several* input features at once.

Key Idea

It extends the straight line to many inputs: each feature gets its own coefficient describing how much it pushes the prediction up or down.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

Real-Life Analogy

Predicting a house price from a single feature is like guessing a dish's taste from *only* the salt. In reality the taste depends on salt *and* spice *and* cooking time together. Multiple regression blends several ingredients, giving each its own weight, to reach a better prediction.

7.1 Example

Predict a house price from *area*, *number of bedrooms*, *number of bathrooms*, and *age* — all at the same time.

7.2 In Practice

The code barely changes — X now has several columns instead of one:

```

1         X = df[["Area", "Bedrooms", "Age"]]    # several input columns
2         y = df["Price"]
3
4         model.fit(X, y)                        # learns one coefficient per
        feature
5
```

7.3 Reading the coefficients

Each coefficient is the expected change in the predicted target for a **one-unit increase in that feature, holding all other features constant**. For example, a coefficient of 1200 on *Area* means each extra unit of area adds about 1200 to the predicted price, assuming bedrooms and age stay the same.

8 Polynomial Regression

What if the data clearly *curves* and no straight line fits well? **Polynomial regression** bends the line by adding powers of the input.

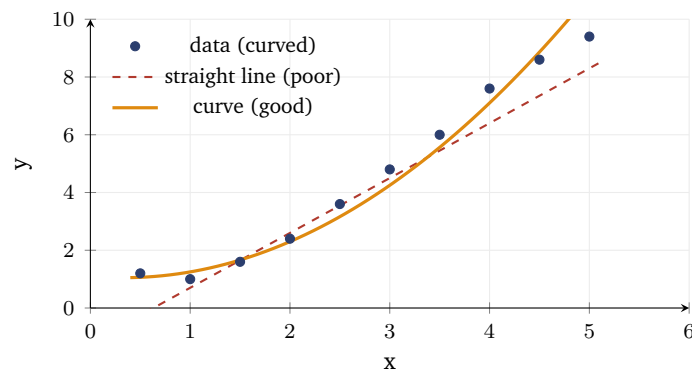
Key Idea

Polynomial regression fits a *curved* line by including features like x^2 , x^3 , and so on. It is still “linear” in the coefficients — we just feed it curved inputs.

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 (+ \beta_3 x^3 + \dots)$$

Real-Life Analogy

A straight ruler cannot trace a winding river. A **flexible plastic ruler** can bend to follow the bends. Each extra power of x gives the curve another bend to play with.



8.1 Examples

Plant growth over time, temperature vs. energy consumption, and fuel efficiency vs. speed all tend to follow curves rather than straight lines.

8.2 In Practice

We first *expand* the features into powers, then fit an ordinary linear model on them:

```

1      from sklearn.preprocessing import PolynomialFeatures
2
3      poly = PolynomialFeatures(degree=2)      # add x^2 terms
4      X_poly = poly.fit_transform(X)          # expand the features
5
6      model.fit(X_poly, y)                    # then fit a linear model as
usual
7

```

Watch Out

Higher degree is *not* always better. A very high degree lets the curve wiggle through every point, fitting the noise instead of the trend — this is **overfitting**, our next topic.

9 Overfitting and Underfitting

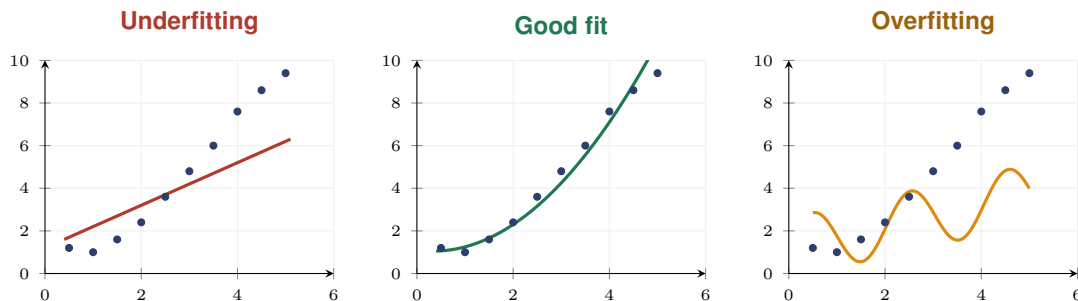
A model can fail in two opposite ways. Understanding both is the key to building models that actually work on new data.

Key Idea

Underfitting: the model is too simple and misses the real pattern — it does badly on both training and new data. **Overfitting**: the model is too complex and memorises the training data (including its noise) — it does great on training data but badly on new data.

Real-Life Analogy

Think of three students preparing for an exam. The **underfitter** barely studied and gives vague answers to everything. The **overfitter** memorised the textbook word-for-word but is stumped the moment a question is phrased differently. The **good student** understood the concepts and can handle new questions. We want the good student.



9.1 How to recognise them

	Training error	Testing error
Underfitting	High	High
Good fit	Low	Low
Overfitting	Very low	High

9.2 Common causes

Underfitting comes from too-simple models or too few useful features. **Overfitting** comes from too-complex models (e.g. a very high polynomial degree), too many features, or too little data.

Try It Yourself

Your model scores $R^2 = 0.98$ on training data but $R^2 = 0.41$ on test data. Which problem is this, and name two things you could try to fix it.

10 The Bias–Variance Tradeoff

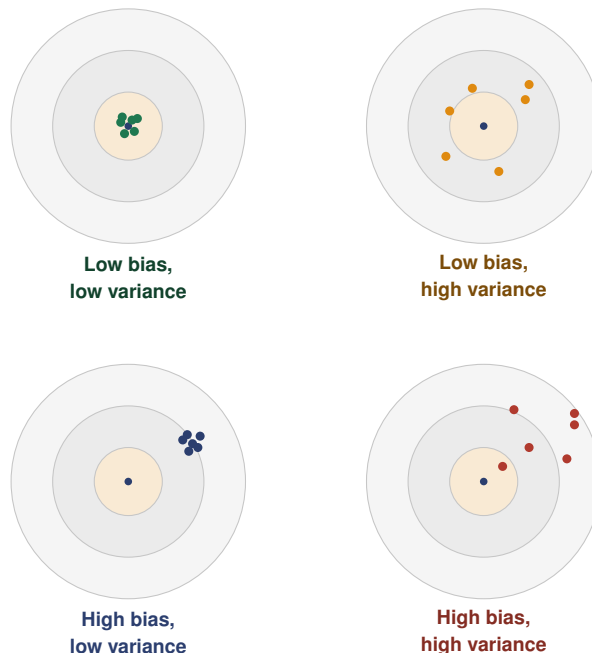
This is the idea that explains *why* overfitting and underfitting happen, and why we can rarely eliminate both at once.

Key Idea

A model's error comes from two competing sources. **Bias** is error from being too simple (wrong assumptions). **Variance** is error from being too sensitive to the particular training data. Lowering one often raises the other — the art is to balance them.

Real-Life Analogy

Picture throwing darts at a bullseye. **High bias** = your darts land tightly together but *off to one side* — consistent, but consistently wrong (too simple). **High variance** = your darts are *scattered all over* — sometimes near the centre, but unreliable (too sensitive). The goal is darts that are both centred and tight.



10.1 Summary

High bias	→	Underfitting (too simple)
High variance	→	Overfitting (too sensitive)
Balanced	→	A model that generalises well

11 Ridge Regression

Ridge is our first tool for *fighting overfitting* in linear models. It does so by gently discouraging large coefficients.

Key Idea

Ridge regression adds an **L2 penalty** to the error: it keeps the usual fit term but also charges a price for big coefficients. This *shrinks* coefficients toward zero (without making them exactly zero) and makes the model more stable.

$$\text{Loss} = \text{MSE} + \lambda \sum_j \beta_j^2$$

Here λ (alpha in code) controls the strength of the penalty: larger λ means more shrinkage.

Real-Life Analogy

Think of a coach telling *every* player on the team to “tone it down a little” so no single player dominates the game. Everyone stays on the team, but their influence is reined in — the team becomes steadier and less likely to be thrown off by one wild performance.

11.1 In Practice

```
1         from sklearn.linear_model import Ridge
2
3         ridge = Ridge(alpha=1.0)           # alpha is the penalty strength (
         lambda)
4         ridge.fit(X_train, y_train)
5
```

When to Use It

You have *many features*, some of them *correlated* with each other, you want to reduce overfitting, and you want to *keep every feature* in the model.

Try It Yourself

You increase alpha from 0.1 to 100. Describe what happens to the size of the coefficients, and what could go wrong if alpha is made far too large.

12 Lasso Regression

Lasso is the close cousin of Ridge — but with a twist that makes it a built-in feature selector.

Key Idea

Lasso regression adds an **L1 penalty**. Unlike Ridge, it can shrink some coefficients *exactly to zero*, effectively **removing** those features. This gives simpler, easier-to-read models.

$$\text{Loss} = \text{MSE} + \lambda \sum_j |\beta_j|$$

Real-Life Analogy

Where the Ridge coach quietly tones everyone down, the Lasso coach is stricter: the least useful players are **benched entirely** (coefficient set to zero). The result is a leaner team made only of the players who genuinely contribute — automatic team selection.

12.1 In Practice

```
1         from sklearn.linear_model import Lasso
2
3         lasso = Lasso(alpha=0.1)           # L1 penalty; can zero-out weak
features    lasso.fit(X_train, y_train)
4
5
```

When to Use It

You have *lots of features* and suspect many are useless, you want *automatic feature selection*, and you value a *simpler, more interpretable* model.

Strengths & Limitations

- ✓ Ridge: keeps all features, very stable with correlated inputs.
- ✓ Lasso: drops weak features automatically, yields simpler models.
- ✗ Both need their `alpha` tuned. ✗ Among correlated features, Lasso may keep just one and drop the rest somewhat arbitrarily.

“Ridge shrinks; Lasso shrinks **and** selects.”

13 Regression Evaluation Metrics

Once a regression model is trained, we must measure *how good its predictions are*. Because the target is a number, we score the model by how far its predictions land from the true values.

Key Idea

Evaluation metrics put a single number on prediction quality. The four standard ones are **MAE**, **MSE**, **RMSE**, and the R^2 **score** — each tells a slightly different part of the story.

Throughout this section, y is the true value, $\hat{y}$ is the prediction, $\bar{y}$ is the mean of the true values, and n is the number of data points. The code assumes you already have `y_test` and `y_pred`.

13.1 Mean Absolute Error (MAE)

The average of the *absolute* gaps between predictions and truth.

$$\text{MAE} = \frac{1}{n} \sum |y - \hat{y}|$$

Real-Life Analogy

MAE is simply: “on average, how many rupees / degrees / units was I off?” If $\text{MAE} = 5$ on house prices measured in lakhs, your predictions are off by about 5 lakh on average. It treats a miss of 10 as exactly twice as bad as a miss of 5 — no drama.

How to read it: lower is better; it is in the *same units* as the target, so it is easy to explain; it is *less affected by extreme outliers* than the squared metrics.

```
1 from sklearn.metrics import mean_absolute_error
2
3 mae = mean_absolute_error(y_test, y_pred)
4
```

13.2 Mean Squared Error (MSE)

The average of the *squared* gaps.

$$\text{MSE} = \frac{1}{n} \sum (y - \hat{y})^2$$

Real-Life Analogy

MSE is the strict examiner: because errors are squared, a big mistake is punished far more heavily than several small ones. A single wildly wrong prediction can dominate the score.

How to read it: lower is better; it *punishes large errors heavily*; its units are the target’s units *squared* (e.g. rupees²), which makes it hard to interpret directly — but it is a favourite *loss function* for training.

```
1 from sklearn.metrics import mean_squared_error
2
3 mse = mean_squared_error(y_test, y_pred)
4
```

13.3 Root Mean Squared Error (RMSE)

Just the square root of MSE.

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Real-Life Analogy

RMSE takes MSE's strictness about big errors but converts the score *back into the original units*, so it is readable again. It is like MAE's stricter sibling: same units, but it cares more about the occasional large miss.

How to read it: lower is better; *same units* as the target (so directly interpretable); still *emphasises larger errors* more than MAE does. This is the most commonly reported regression metric.

```
1 import numpy as np
2 from sklearn.metrics import mean_squared_error
3
4 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
5
```

13.4 R^2 Score (Coefficient of Determination)

R^2 measures the *proportion of the variation* in the target that the model manages to explain.

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Real-Life Analogy

R^2 is like a report-card percentage for the model, compared against a lazy baseline that always guesses the average. $R^2 = 0.8$ roughly means “the model explains 80% of what’s going on; the remaining 20% it cannot account for.”

How to read it:

- $R^2 = 1$ ➤ perfect predictions.
- $R^2 = 0$ ➤ no better than always predicting the mean.
- $R^2 < 0$ ➤ worse than predicting the mean (a genuinely bad model).

Higher is generally better, but always read R^2 *alongside* an error metric like MAE or RMSE.

```
1 from sklearn.metrics import r2_score
2
3 r2 = r2_score(y_test, y_pred)
4
```


13.5 Which metric should I use?

Metric	Units	Use it when. . .
MAE	Same as target	you want a simple, outlier-tolerant average error.
MSE	Target squared	you are training and want to punish big errors.
RMSE	Same as target	you want a readable error that still flags big misses.
R^2	None (0–1)	you want an overall “how much is explained” score.

Watch Out

Never judge a regression model on R^2 alone. A model can have a respectable R^2 while still making errors that are far too large for your real-world use. Report R^2 and RMSE (or MAE) together.

Try It Yourself

For a house-price model (prices in lakhs) you get $\text{MAE} = 3$, $\text{RMSE} = 8$, and $R^2 = 0.86$. (a) What does RMSE being much larger than MAE suggest about the errors? (b) In one sentence, how good is this model?